01 / TON POINT DE DÉPART

Sentinel Lab
Guide de démonstration

Présentation du **31 octobre 2026** • Prototype local sur Windows

LE SUJET À ANNONCER

Détection en temps réel d'une surcharge HTTP sur une API locale à l'aide d'un autoencodeur.

Ce que tu vas démontrer

Une API reçoit de vraies requêtes. Un collecteur mesure son comportement chaque seconde environ. Un réseau de neurones calcule un score d'anomalie et le tableau de bord affiche les alertes. Le scénario d'attaque est une surcharge locale contrôlée de 25 secondes.

Avant de parler	Action à effectuer
Ouvrir le prototype	Dans le dossier **sentinel-lab**, double-cliquer sur **DEMARRER.bat**. Garder la fenêtre du lanceur ouverte.
Ouvrir le tableau de bord	http://127.0.0.1:8765 Le modèle sauvegardé se charge automatiquement.
Préparer le modèle	Si les conditions du PC ont changé, cliquer sur **Calibrer le modèle**. Prévoir environ deux minutes, avant le passage devant le jury.
Vérifier le point de départ	Trafic normal actif, mesures qui évoluent, modèle entraîné, pas d'alerte persistante. Revenir en haut de la page.

TA PHRASE D'INTRODUCTION

« Mon projet détecte des comportements inhabituels sur une API. Je vais montrer le trafic normal, provoquer une surcharge dans ce laboratoire, puis observer la réponse du réseau de neurones. »

Préparation : environ 2 minutes si recalibration. **Démonstration :** 4 minutes. **Questions :** garder les pages 6 et 7 sous la main.

Ce PC dispose déjà des dépendances. Le prototype peut fonctionner sans Internet. Une copie sur un autre PC demande une installation et une nouvelle vérification avant la soutenance.

02 / DÉROULÉ - PREMIÈRE PARTIE

Commencer par le normal

Le jury doit comprendre ce qui est observé avant de voir l'attaque. Répète cette séquence une fois avant la présentation.

1. 0:00 à 0:30 - Présenter l'objectif

Tu fais : Cliquer sur **Trafic normal**, puis montrer les quatre indicateurs en haut.

Tu montres : Le débit et la latence changent réellement. Les erreurs HTTP restent normalement faibles.

Tu dis : « Le client envoie de vraies requêtes HTTP à mon API locale. Le système observe ses réponses et mesure son comportement. »

2. 0:30 à 1:00 - Montrer la chaîne complète

Tu fais : Faire défiler jusqu'aux courbes, puis aux dernières réponses de l'API.

Tu montres : Les graphiques, la route **/api/service**, les codes HTTP et les durées.

Tu dis : « Les données affichées proviennent des requêtes effectuées maintenant. Les commandes du tableau de bord ne sont pas incluses dans le trafic analysé. »

3. 1:00 à 1:45 - Identifier le deep learning

Tu fais : Ouvrir visuellement la partie **Le deep learning, en clair**. Montrer les couches et les courbes d'apprentissage.

Tu montres : Le réseau **6 → 12 → 6 → 2 → 6 → 12 → 6**, ses **356 paramètres** et les pertes enregistrées pendant l'entraînement.

Tu dis : « L'autoencodeur apprend à reconstruire le trafic normal. Il compresse six mesures en deux valeurs puis tente de retrouver les six mesures initiales. Il apprend ses poids avec PyTorch. »

> **LA DISTINCTION À BIEN DIRE**
>
> « L'apprentissage a lieu avant la démonstration. Pendant la démonstration, les poids sont fixes : le modèle analyse de nouvelles mesures en direct. C'est l'inférence. »

Si le jury veut voir l'entraînement

Cliquer sur **Calibrer le modèle** et annoncer environ deux minutes supplémentaires. Le système collecte 90 fenêtres normales, entraîne le réseau, puis sauvegarde les poids. Ne pas lancer de surcharge pendant cette étape. Les boutons de scénario sont désactivés jusqu'à la fin.

Une courbe d'apprentissage qui diminue montre que le réseau améliore sa reconstruction. Elle ne démontre pas à elle seule qu'il détecte toutes les attaques.

03 / DÉROULÉ - SECONDE PARTIE

Déclencher. Observer. Expliquer.

Reste sur le tableau de bord. N’annonce pas une alerte avant de l’avoir vue apparaître.

4. 1:45 à 2:30 - Lancer la surcharge

Tu fais : Cliquer une seule fois sur **Lancer la surcharge**. Le scénario s’arrête au bout de 25 secondes.

Tu montres : La hausse du débit, les latences, les éventuels HTTP 503, puis le score et l’état de détection.

Tu dis : « Je déclenche une surcharge contrôlée sur cette seule API. Sa capacité est limitée à trois traitements simultanés pour rendre la dégradation visible sans saturer tout l’ordinateur. »

5. Pendant l’alerte - Relier score et décision

Tu fais : Pointer **Score / seuil**, puis le message d’alerte et le délai mesuré.

Tu montres : Une valeur supérieure à 1 indique une fenêtre suspecte. Deux fenêtres consécutives confirment l’alerte.

Tu dis : « Le modèle reconstruit mal ces nouvelles mesures. Le score franchit le seuil et l’anomalie est confirmée. Le détecteur reçoit uniquement les mesures, jamais l’information que j’ai appuyé sur le bouton. »

6. 2:30 à 3:00 - Montrer la récupération

Tu fais : Laisser le scénario se terminer, ou cliquer sur **Trafic normal** pour l’interrompre.

Tu montres : Les métriques se stabilisent et le score revient sous le seuil. Le journal conserve les événements.

Tu dis : « Lorsque la surcharge s’arrête, l’API retrouve son comportement habituel. La décision suit les mesures observées. »

7. 3:00 à 4:00 - Faire le contrôle et conclure

Tu fais : Cliquer sur **Pic légitime** pendant 25 secondes, puis sur **Exporter CSV**.

Tu montres : Une charge de 24 requêtes/s demandées, les éventuelles alertes et le fichier de mesures.

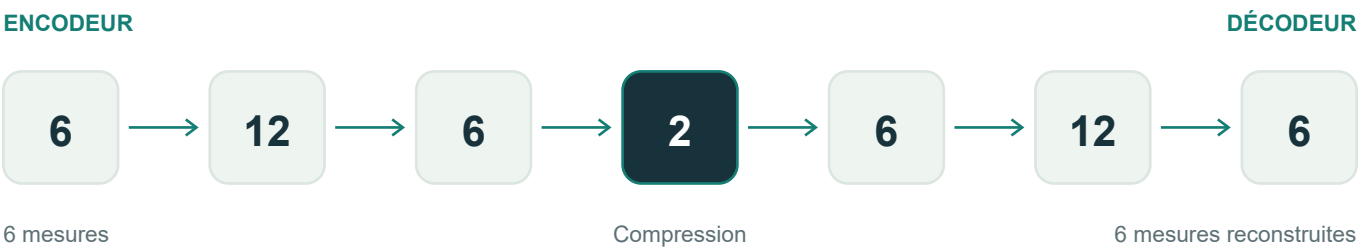
Tu dis : « Ce contrôle teste une hausse de trafic qui fait partie du fonctionnement prévu. Le résultat doit être observé : un trafic inhabituel n’est pas toujours une attaque. »

> **POUR TERMINER**
>
> « J’ai réalisé une chaîne complète : API, génération de trafic, collecte, réseau de neurones et alertes. Les résultats sont encourageants dans ce laboratoire. Il reste à les évaluer sur davantage de charges normales et de scénarios d’attaque. »

04 / COMPRENDRE TON MODÈLE

Le rôle de l’autoencodeur

Un autoencodeur est un réseau de neurones qui apprend à produire une reconstruction de ce qu’on lui donne en entrée.



Étape	Ce qui se passe
1. Apprendre le normal	90 fenêtres mesurées sur l’API : **67 pour l’entraînement**, puis **23 pour la validation**, dans l’ordre chronologique. Aucune attaque n’est utilisée pour entraîner le réseau.
2. Ajuster les poids	Le réseau compare les six entrées à leurs reconstructions. La rétropropagation et l’optimiseur **Adam** modifient les poids pour réduire l’erreur moyenne au carré, appelée **MSE**.
3. Surveiller en direct	De nouvelles mesures traversent le même réseau. Une erreur élevée indique qu’elles sont éloignées de ce qu’il a appris à reconstruire.

Les six mesures en entrée

Mesure	Ce qu’elle décrit
Débit	Nombre de réponses terminées par seconde.
Latence p95	Durée sous laquelle se situent 95 % des réponses.
Taux d’erreur	Part des réponses HTTP 4xx et 5xx.
Latence moyenne	Durée moyenne des requêtes, attente côté API comprise.
Concurrence maximale	Maximum de requêtes simultanées dans la fenêtre.
Octets par seconde	Volume des corps de réponses par seconde.

Pourquoi du deep learning ? Le modèle possède plusieurs couches cachées de neurones, des activations non linéaires **Tanh** et des poids appris par rétropropagation. Le framework PyTorch exécute réellement l’entraînement et l’inférence.

Analogie à dire : « Il apprend à résumer puis à reconstruire une situation habituelle. Quand il reconstruit mal une nouvelle situation, nous la considérons comme suspecte. Cela reste une hypothèse à vérifier, pas une preuve d’intrusion. »

05 / LIRE LES RÉSULTATS

Ce que les chiffres veulent dire

Cette capture a été prise pendant un véritable scénario local du 30 octobre 2026. Elle sert de repère ; les valeurs de demain pourront être différentes.



Débit et latence

Ils décrivent les effets de la charge. Un débit élevé ne suffit pas à prouver une attaque.

Erreurs HTTP

Un code 503 signifie que notre API ne peut pas prendre en charge une requête dans le délai prévu.

Score / seuil

1 est la limite. 2 signifie deux fois le seuil, et non « 200 % de probabilité d'attaque ».

Alerte confirmée

Elle exige deux fenêtres suspectes consécutives. Une fenêtre isolée peut être indiquée comme suspecte sans confirmation.

Résultat du test de fonctionnement sauvegardé

Séquence observée	Fenêtres examinées	Avec alerte
Trafic normal	14	0
Pic légitime	22	0
Surcharge	22	22
Récupération	12	0

Délai de détection observé : 1,5 s. Le modèle utilisé par ce test est celui sauvegardé le 30 octobre 2026 à 18:43 UTC.

Ces nombres décrivent un petit test local de fonctionnement. Les 22 fenêtres avec alerte ne sont pas 22 attaques indépendantes. Ils ne permettent pas d'annoncer « 100 % de précision » ou un taux général de fausses alertes.

Source : **artifacts/validation_live.json**, test du 30 octobre 2026 à 18:46 UTC. Le script retient des fenêtres après les premières secondes de transition ; les 25 secondes du scénario ne sont pas toutes comptées.

06 / PRÉPARER LES QUESTIONS

Répondre au jury sans surpromettre

Réponds d’abord simplement, puis donne un détail technique si le jury le demande.

Pourquoi un autoencodeur ?

Il peut apprendre une référence à partir du trafic normal, sans avoir besoin de toutes les attaques pour son entraînement. Cela ne garantit pas qu’il détectera chaque attaque inconnue.

Où est le deep learning ?

Dans detector.py : un réseau PyTorch à plusieurs couches, avec 356 paramètres appris. Je peux montrer l’architecture, les courbes de perte et les poids sauvegardés.

Quelles données as-tu utilisées ?

Les données normales collectées sur cette API locale : 90 fenêtres, dont 67 d’entraînement et 23 de validation. CICIoT2023 n’est pas utilisé dans cette version.

Est-ce vraiment en direct ?

Oui : les clients envoient de vraies requêtes maintenant, et les métriques sont recalculées environ chaque seconde. Le modèle était entraîné avant la démo ; son inférence s’exécute en direct.

Est-ce un DDoS, de l’IoT ou de la 5G ?

Cette version démontre un DoS HTTP local avec un seul générateur. Elle n’implémente pas un botnet distribué, un parc IoT réel ou un cœur 5G.

Une forte fréquentation peut-elle déclencher une alerte ?

Oui. Une surcharge légitime peut ressembler à une attaque. Le pic légitime de la démonstration est un contrôle limité ; il faut tester beaucoup plus de profils normaux pour évaluer ce risque.

Pourquoi ne pas utiliser un simple seuil de débit ?

Ce modèle combine six mesures. Cependant, je n’ai pas encore démontré qu’il surpasse un seuil simple ou Isolation Forest. Une comparaison sur les mêmes données est une prochaine étape.

Le système bloque-t-il l’attaque ?

Non. Le prototype mesure et alerte. Les HTTP 503 viennent de la limite de capacité de l’API, pas d’un blocage décidé par le réseau de neurones.

TA CONTRIBUTION

« J’ai intégré la génération de trafic, la collecte, l’apprentissage, l’inférence et la visualisation. Mon apport est un prototype reproductible et une évaluation locale documentée. »

07 / À GARDER SOUS LA MAIN

Réglages, secours et dernière vérification

Si cela arrive…	Que faire et que dire
La page ne s’ouvre pas	Relancer **DEMARRER.bat**, garder le lanceur ouvert, puis ouvrir **http://127.0.0.1:8765**. Attendre quelques secondes au démarrage de PyTorch.
Le bouton de surcharge est grisé	Attendre la fin de la calibration ou du scénario actuel. Une calibration complète dure environ deux minutes.
Alerte pendant le trafic normal	Arrêter les scénarios, laisser le PC se stabiliser et recalibrer. Dire : « Le comportement observé diffère de ma référence ; je dois vérifier la calibration. »
Pas d’alerte pendant la surcharge	Vérifier les mesures, le scénario et le score. Ne pas annoncer une détection inexistante. Recalibrer avant un nouvel essai si les conditions ont changé.
Il faut interrompre la démo	**Trafic normal** interrompt la surcharge tout en continuant à générer du trafic normal. **Arrêter le trafic** coupe le générateur. **Ctrl+C** dans le lanceur ferme le serveur.

Si une question technique approfondie arrive

Prétraitement : log(1+x), puis standardisation apprise sur les 67 fenêtres d’entraînement uniquement. Les échelles ont un plancher pour éviter d’amplifier les variations minuscules.

Erreur : MSE = moyenne des 6 différences au carré entre entrées normalisées et reconstructions. Les contributions affichées décomposent cette erreur ; elles n’expliquent pas à elles seules la cause d’une attaque.

Seuil : max(2,5 × quantile 99 % des scores de validation normale, 1,5). Le facteur 2,5 et le plancher 1,5 sont des choix prudents de démonstration, pas des garanties statistiques.

Entraînement : Adam, activations Tanh, MSE, au plus 300 époques et arrêt anticipé. Le modèle livré a effectué 300 époques.
Décision : deux fenêtres consécutives au-dessus du seuil.

Dernière vérification avant de passer

PC branché et sessions inutiles fermées • même machine que pour la calibration • API et modèle chargés • essai complet déjà répété • navigateur en haut du tableau de bord • ce guide disponible hors ligne.

Garder le dossier **sentinel-lab** et l’environnement **work/venv** en place sur ce PC. Le ZIP contient le code et le modèle, mais pas Python ni ses bibliothèques. Le fichier **session.csv** est réinitialisé au redémarrage : exporter les mesures à conserver.

Fichiers de référence

app.py : API et collecte. **detector.py** : réseau et score. **artifacts/model.json** : entraînement. **artifacts/validation_live.json** : test. **README.md** : guide technique complet.

Ce guide décrit la version locale vérifiée le 30 octobre 2026. Les résultats présentés proviennent des fichiers du prototype et de sa capture de démonstration ; aucun benchmark externe n’est revendiqué.